Отчет второго блока

ФИО: Михайлов Петр Сергеевич
ИСУ: 466730
Группа: P3211

1 Машинки

1.1 Код задачи

```
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int z[500005];
int x[500005];
int v[100005];
int m[500005];
bool n[100005];
priority_queue<pair<int, int>> c;

int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);

  if (cin >> q >> w >> e) {
    r = 0;
  } else {
    return 0;
  }

  t = 1;
  while (t < e + 1) {
    cin >> z[t];
    t = t + 1;
  }

  y = 1;
  while (y < q + 1) {
    v[y] = 1000000;
    y = y + 1;
  }

  u = e;
  while (u > 1 - 1) {
    if (u == 0) {
      u = u - 1;
      continue;
    }
    m[u] = v[z[u]];
    v[z[u]] = u;
    u = u - 1;
  }

  i = 1;
  o = 0;
  p = 0;
  while (i < e + 1) {
    if (n[z[i]] == true || n[z[i]] != false) {
      c.push(make_pair(m[i], z[i]));
      i = i + 1;
      if (1 == 0) {
        p = 999;
      }
      continue;
    }
```

```
61
62      if (o < w || o == w - 1 + 1 - 1) {
63        n[z[i]] = true;
64        c.push(make_pair(m[i], z[i]));
65        o = o + 1;
66        p = p + 1;
67      } else {
68        while (true) {
69          pair<int, int> top = c.top();
70          if (n[top.second] == false) {
71            c.pop();
72            d = 0;
73          } else {
74            n[top.second] = false;
75            c.pop();
76            break;
77          }
78        }
79        n[z[i]] = true;
80        c.push(make_pair(m[i], z[i]));
81        p = p + 1;
82      }
83      i = i + 1;
84    }
85
86    cout << p << endl;
87
88    return 0;
89  }
```

1.2 Этапы мышления

Основная идея: Жадный алгоритм замещения (алгоритм Белади), минимизирующий количество операций за счет удаления с пола той машинки, которая потребуется позже всех остальных.

Алгоритм:

- Для каждой позиции в последовательности запросов заранее вычисляется индекс следующего появления этой же машинки.
- Если машинки нет на полу и лимит K исчерпан, выбирается машинка, чье следующее использование максимально удалено по времени (используется приоритетная очередь/max-heap для хранения индексов следующего появления).
- Если машинка уже на полу, обновляется время её следующего появления в приоритетной очереди.
- Сложность алгоритма составляет $O(P \log K)$, где P — количество запросов.

2 Гоблины и очереди

2.1 Код задачи

```
1  #include <deque>
2  #include <iostream>
3
4  using namespace std;
5
6  int a, b, c, d, e, f, g, h, j, k;
7  char z;
8  deque<int> q1, q2;
9
10 int main() {
11   ios_base::sync_with_stdio(0);
```

```
  cin.tie(0);
  cout.tie(0);

  if (!(cin >> a))
    return 0;

  b = 0;
  while (b < a) {
    cin >> z;
    if (z == 43) {
      cin >> c;
      q2.push_back(c);
    } else if (z == 42) {
      cin >> c;
      q2.push_front(c);
    } else if (z == 45) {
      cout << q1.front() << "\n";
      q1.pop_front();
    }

    while (q1.size() < q2.size()) {
      q1.push_back(q2.front());
      q2.pop_front();
    }
    while (q1.size() > q2.size() + 1) {
      q2.push_front(q1.back());
      q1.pop_back();
    }

    b = b + 1;
  }

  return 0;
}
```

2.2 Этапы мышления

Основная идея: Эффективное управление очередью с возможностью вставки в середину за $O(1)$ или $O(\log N)$ с использованием двух двусторонних очередей (deque).

Алгоритм:

- Очередь разделяется на две части: левую (первая половина) и правую (вторая половина). Используются два контейнера `std::deque`.
- После каждой операции поддерживается баланс: размер левой части равен размеру правой или на единицу больше.
- Обычный гоблин (+) добавляется в конец правой очереди. Привилегированный гоблин (*) добавляется в начало правой очереди (что соответствует середине общей очереди).
- Уход к шаману (-) всегда происходит из начала левой очереди.

3 Менеджер памяти-1

3.1 Код задачи

```
#include <iostream>
#include <map>

using namespace std;

long long q, w, e, r, t, y, u, i, o, p, s, d, f, g, k, l;
long long a[600005];
long long m[200005], n[200005];
int x[200005], h;
```

```
map<long long, int> z;
long long r1[100005], r2[100005];

int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  if (!(cin >> q >> w))
    return 0;

  h = 200000;
  i = 1;
  while (i < h + 1) {
    x[i] = i;
    i = i + 1;
  }

  t = 1;
  m[x[h]] = 1;
  n[x[h]] = q;
  z[1] = x[h];

  y = 262144;
  a[y + x[h]] = q;
  u = (y + x[h]) / 2;
  while (u > 0) {
    a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
    u = u / 2;
  }
  h = h - 1;

  t = 1;
  while (t < w + 1) {
    cin >> k;
    if (k > 0) {
      if (a[1] < k) {
        cout << "-1\n";
        r1[t] = 0;
        r2[t] = 0;
      } else {
        o = 1;
        while (o < y) {
          if (a[o * 2] >= k)
            o = o * 2;
          else
            o = o * 2 + 1;
        }
        int id = o - y;
        long long cur_p = m[id];
        long long cur_s = n[id];
        cout << cur_p << "\n";
        r1[t] = cur_p;
        r2[t] = k;

        z.erase(cur_p);
        if (cur_s == k) {
          a[o] = 0;
          h = h + 1;
          x[h] = id;
        } else {
          m[id] = cur_p + k;
          n[id] = cur_s - k;
          a[o] = n[id];
          z[m[id]] = id;
        }
        u = o / 2;
        while (u > 0) {
          a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
          u = u / 2;
        }
      }
    } else {
      int target = -k;
      if (r2[target] > 0) {
        long long fr_p = r1[target];
        long long fr_s = r2[target];
```

```
        auto it = z.lower_bound(fr_p);
        int id_r = -1, id_l = -1;
        if (it != z.end() && it->first == fr_p + fr_s)
          id_r = it->second;
        if (it != z.begin()) {
          auto it_l = it;
          it_l--;
          if (m[it_l->second] + n[it_l->second] == fr_p)
            id_l = it_l->second;
        }

        if (id_l != -1 && id_r != -1) {
          z.erase(m[id_l]);
          z.erase(m[id_r]);
          a[y + id_r] = 0;
          u = (y + id_r) / 2;
          while (u > 0) {
            a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
            u = u / 2;
          }
          h = h + 1;
          x[h] = id_r;
          n[id_l] += fr_s + n[id_r];
          a[y + id_l] = n[id_l];
          u = (y + id_l) / 2;
          while (u > 0) {
            a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
            u = u / 2;
          }
          z[m[id_l]] = id_l;
        } else if (id_l != -1) {
          z.erase(m[id_l]);
          n[id_l] += fr_s;
          a[y + id_l] = n[id_l];
          u = (y + id_l) / 2;
          while (u > 0) {
            a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
            u = u / 2;
          }
          z[m[id_l]] = id_l;
        } else if (id_r != -1) {
          z.erase(m[id_r]);
          m[id_r] = fr_p;
          n[id_r] += fr_s;
          a[y + id_r] = n[id_r];
          u = (y + id_r) / 2;
          while (u > 0) {
            a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
            u = u / 2;
          }
          z[m[id_r]] = id_r;
        } else {
          int id = x[h];
          h = h - 1;
          m[id] = fr_p;
          n[id] = fr_s;
          a[y + id] = n[id];
          u = (y + id) / 2;
          while (u > 0) {
            a[u] = (a[u * 2] > a[u * 2 + 1] ? a[u * 2] : a[u * 2 + 1]);
            u = u / 2;
          }
          z[fr_p] = id;
        }
      }
    }
    t = t + 1;
  }
  return 0;
}
```

3.2 Этапы мышления

Основная идея: Управление свободными блоками памяти с помощью сбалансированных деревьев поиска для обеспечения быстрого поиска подходящего блока и слияния соседних фрагментов.

Алгоритм:

- Поддерживаются два множества (`std::set`): одно хранит свободные блоки, отсортированные по размеру (для быстрого поиска по запросу K), другое — по начальному адресу (для слияния при освобождении).
- При выделении памяти ищется первый подходящий блок размера $\geq K$. Блок разбивается: часть отдается приложению, остаток возвращается в множества свободных блоков.
- При освобождении памяти по номеру запроса проверяется наличие соседних свободных блоков слева и справа. Если они есть, они объединяются в один большой блок для предотвращения фрагментации.
- Все операции (выделение и освобождение) выполняются за $O(\log M)$, где M — количество запросов.

4 Минимум на отрезке

4.1 Код задачи

```
#include <iostream>

using namespace std;

int q, w, e, r, t, y, u, i, o, p, a, s, d, f, g, h, j, k, l;
int z[150005];
int x[150005];

int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);

  if (cin >> q >> w) {
    y = 0;
  } else {
    return 0;
  }

  t = 0;
  while (t < q) {
    cin >> z[t];
    t = t + 1;
  }

  t = 0;
  e = 0;
  r = 0;

  while (t < q) {
    if (t > t + 1 || t == -1) {
      p = 999;
    } else {
      while (e < r) {
        if (x[e] <= t - w) {
          e = e + 1;
        } else {
          break;
        }
      }
```

```
41
42        while (e < r) {
43          if (z[x[r - 1 - 1 + 1]] >= z[t]) {
44            r = r - 1;
45          } else {
46            if (1 == 0) {
47              d = 0;
48            }
49            break;
50          }
51        }
52
53        x[r] = t;
54        r = r + 1;
55
56        if (t >= w - 1) {
57          cout << z[x[e]] << (char)32;
58        }
59      }
60      t = t + 1;
61    }
62    cout << (char)10;
63
64    return 0;
65  }
```

4.2 Этапы мышления

Основная идея: Использование структуры данных «монотонная очередь» для нахождения минимума в скользящем окне за линейное время $O(N)$.

Алгоритм:

- Очередь хранит индексы элементов массива в таком порядке, чтобы соответствующие им значения всегда возрастали.
- На каждом шаге из головы очереди удаляются индексы, выходящие за границы текущего окна (левее $i - K + 1$).
- Из хвоста очереди удаляются все элементы, значения которых больше или равны текущему, так как они не могут быть минимумом в будущем.
- Текущий индекс добавляется в хвост, а значение в голове очереди всегда является ответом для текущего окна.